On Universal Models

Document Number:	P4034R0
Date:	2026-03-08
Audience:	All of WG21
Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: March 2026

1. Disclosure

2. Which Universal Models Succeed?

3. A Universal Model in Plain Sight?

4. Which Pattern Does `std::execution` Follow?

5. The Cost of Foreclosure

6. Conclusion

References

Abstract

This paper examines the historical record of universal models in computing and asks three questions:

1. Which universal models succeed, and which fail?
2. Has a universal async protocol already emerged in C++ without anyone designing it?
3. Which pattern does `std::execution` follow?

Revision History

R0: March 2026

- Initial revision.

---

1. Disclosure

The author has papers before the committee proposing coroutine-based I/O (P4003) and analyzing `std::execution` (P2583, P4007, P4014). Every claim in this paper is sourced to public committee records, published scholarship, or cited implementations.

2. Which Universal Models Succeed?

Model	Scope	Emergence	Adoption before standardization	Outcome
TCP/IP	Narrow (IP datagram)	DARPA implementations	Worldwide	Won over OSI
IEEE 754	Narrow (one FP format)	Intel 8087	Every vendor	Won over chaos
STL iterators	Narrow (3 operations)	STL library, 1994	Every container	Universal
REST	Narrow (resources and verbs)	HTTP practice	The web	Won over CORBA
CORBA	Wide (6+ bundled concerns)	OMG committee	Vendor mandates	Failed
OSI	Wide (7 layers)	ISO committee	Government mandates	Failed

Butler Lampson:

> *"An interface should capture the minimum essentials of an abstraction. Don't generalize; generalizations are generally wrong."*
>
> *Lampson, "Hints for Computer System Design"[1] (1983)*

Ted Kaminski captured the tradeoff:

> *"An all-powerful abstraction is a meaningless one (you've just got a new word for 'thing')."*
>
> *Kaminski, "The One Ring Problem"[2] (2018)*

3. A Universal Model in Plain Sight?

C++20 coroutines define an awaitable protocol: three operations that any type can satisfy.

```
await_ready()     -> bool
await_suspend(h) -> void | bool | coroutine_handle<>
await_resume()    -> T
```

Property	Iterators	Awaitable Protocol
Operations	3	3
Cross-library interop	Emergent	Emergent
Designed for interop?	No	No
Type erasure	N/A	`coroutine_handle<>` built in
Practice before standard	STL, 1994	MSVC, Clang, GCC before C++20
Stable since	C++98 (28 years)	C++20 (6 years)

A `task<T>` from one library can be `co_await`ed inside a `task<T>` from another library. No coordination between the library authors is required. The following combinations work today:

Library A	Library B	Coordination required
cppcoro::task	folly::coro::Task	None
asio::awaitable	custom task type	None

Library A	Library B	Coordination required
tmc::task (TooManyCooks)	any coroutine	None
stdexec sender (via task)	any awaitable	None

The structure mirrors the Internet's hourglass architecture[3]. Below the waist: runtimes (Asio, libuv, io_uring, GPU schedulers). Above the waist: task types, algorithms, ergonomic layers. At the waist: three operations. Innovation happens above and below. The waist is fixed.

D4003[4] extends the protocol with an `io_env` parameter that carries stop token, executor, and allocator through `await_suspend` - context propagation without coupling the awaitable to any promise type:

```
template<typename A>
concept IoAwaitable =
    requires(
        A a, std::coroutine_handle<> h, io_env const* env )
    {
        a.await_suspend( h, env );
    };
```

4. Which Pattern Does `std::execution` Follow?

	CORBA	`std::execution`
Designed by	Standards consortium (OMG)	Standards committee (WG21)
Scope	Universal distributed object middleware	Universal async execution model
Bundled concerns	Lifecycle, naming, transactions, security, concurrency, IDL	Scheduling, context propagation, error handling, cancellation, algorithm dispatch, hardware backends
Companion specs in flight	CCM, security service, transaction service	P2079, P3164, P3373, P3388, P3425, P3481, P3552, P3557, P3564, P3826
Competing narrow alternatives	REST/HTTP, EJB, then microservices	Asio, folly, Seastar, TooManyCooks, Taskflow
Primary use case deferred	Web integration (arrived too late)	Networking (deferred to C++29)

	CORBA	`std::execution`
Institutional backing	ISO, telecom vendors, government contracts	WG21, NVIDIA, national body votes

Michi Henning, on the CORBA Component Model:

> *"The specification was large and complex and much of it had never been implemented, not even as a proof of concept."*
>
> *Henning, "The Rise and Fall of CORBA"[5], ACM Queue (2006)*

David Chappell, writing in 1998 while CORBA was still considered viable:

> *"Is it even possible for committees to successfully create new technology? History is not encouraging."*
>
> *Chappell, "The Trouble With CORBA"[6] (1998)*

On 2021-09-28, SG1 polled:

> *"We believe we need one grand unified model for asynchronous execution in the C++ Standard Library, that covers structured concurrency, event based programming, active patterns, etc."*

The result was **no consensus** (leaning in favor): 4 SF, 9 WF, 5 N, 5 WA, 1 SA (P2453R0)[7]. The committee did not achieve consensus that a universal model was needed. The direction proceeded as a mandate without one.

Domain	Evidence	Status
GPU dispatch	nvexec, CUDA extensions	Proven
HFT pipelines	Leahy CppCon 2025[8], stdexec	Proven
Networking	"Explored for C++29"[9]	Deferred
Type erasure	libunifex #244[10] (5 years open)	Structural gap
Ordinary developer	see code below	Underserved

What the ordinary developer wants to write:

```
auto [ec, n] = co_await stream.read(buf);
```

What `std::execution` requires:

```
auto s = just(buf)
       | let_value([&](auto b) {
           return async_read(stream, b);
         })
       | then([](std::size_t n) { /* ... */ });
sync_wait(on(system_ctx.get_scheduler(), std::move(s)));
```

`std::execution` abstracts over scheduling, context propagation, error handling, cancellation, algorithm dispatch, and hardware backend selection. Iterators abstract over traversal.

5. The Cost of Foreclosure

The cost of a premature universality claim is not borne by the model. It is borne by every domain the model was not designed for.

When models compete	When universality is mandated
Each domain optimizes for its own constraints	Every domain pays for one domain's constraints
The committee standardizes what practitioners adopt	Practitioners adopt what the committee standardizes
Universality is discovered through evidence	Universality is declared by vote
Alternatives exist to absorb the cases that do not fit	Alternatives are foreclosed before they are explored

Every major programming language ships standard networking: Python[11], Java[12], Go[13], Rust[14], C#[15], JavaScript[16]. None ships a sender/receiver execution framework. C++ ships neither.

6. Conclusion

The executor discussion began in 2012. Fourteen years later, C++ has no standard networking, yet has `std::execution` with 50 post-approval items since Tokyo including two Priority 1 safety defects[17], shipping in C++26 without the use case that started the discussion.

Every successful model in Section 2 earned its moniker through competition. No poll foreclosed alternatives to iterators.

Universal models that endure have one thing in common: they earned the name.

References

1. Butler Lampson. ‘‘Hints for Computer System Design.’’ 1983. http://research.microsoft.com/en-us/um/people/blampson/33-Hints/Acrobat.pdf
2. Ted Kaminski. ‘‘The One Ring Problem.’’ 2018. https://tedinski.com/2018/01/30/the-one-ring-problem-abstraction-and-power.html
3. Wikipedia. ‘‘Hourglass model.’’ https://en.wikipedia.org/wiki/Hourglass_model
4. D4003. Falco et al. ‘‘IoAwaitables: A Coroutines-Only Framework.’’ WG21. https://wg21.link/p4003
5. Michi Henning. ‘‘The Rise and Fall of CORBA.’’ ACM Queue, 2006. https://dl.acm.org/doi/10.1145/1142031.1142044
6. David Chappell. ‘‘The Trouble With CORBA.’’ 1998. https://davidchappell.com/writing/article_Trouble_CORBA.php
7. P2453R0. ‘‘2021 October Library Evolution and Concurrency Networking and Executors Poll Outcomes.’’ WG21, 2022. https://wg21.link/p2453
8. Robert Leahy. ‘‘std::execution in Asio Codebases: Adopting Senders Without a Rewrite.’’ CppCon 2025. https://cppcon2025.sched.com/event/27bQ1
9. nvidia.github.io/stdexec. ‘‘Standardization Status (as of 2025).’’ https://nvidia.github.io/stdexec/
10. libunifex issue #244. ‘‘Question about any_sender_of usage.’’ 2021. https://github.com/facebookexperimental/libunifex/issues/244
11. Python standard library: socket. https://docs.python.org/3/library/socket.html
12. Java standard library: java.net. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/package-summary.html
13. Go standard library: net. https://pkg.go.dev/net
14. Rust standard library: std::net. https://doc.rust-lang.org/std/net/
15. C# standard library: System.Net.Sockets. https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets

16. Node.js standard library: net. https://nodejs.org/api/net.html

17. LWG unresolved prioritized list. https://cplusplus.github.io/LWG/unresolved-prioritized.html